

Robust Supervisors for Intersection Collision Avoidance in the Presence of Legacy Vehicles

Heejin Ahn* , Andrea Rizzi, and Alessandro Colombo

Abstract: Vehicle coordination at road intersections to prevent collisions has recently drawn a lot of research attention, but still remains challenging due to a large volume of traffic composed of autonomous and human-driven vehicles. In this paper, we present the design and validation of a supervisory algorithm for collision avoidance at road intersections, in the simultaneous presence of measurement errors, unmodeled dynamics, and vehicles that are not equipped with autonomous driving features. We design a supervisor that takes control of vehicles that are capable of communication and autonomous actions, only when necessary to avoid a collision, or otherwise leaves control to their drivers. This supervisor is least restrictive, that is, it takes control away from drivers only when necessary to avoid future collisions. Since the complexity of the supervisor algorithm is combinatorial with the number of vehicles, we also design an approximate supervisor that can handle more realistic scenarios with a larger number of vehicles in polynomially bounded time at the expense of an additional, yet quantified, restrictiveness. The least restrictive supervisor is validated through experimental testings using three small-scale vehicles, and the approximate supervisor is validated through computer simulations.

Keywords: Collision avoidance, intersection coordination, robust control, safety verification

1. INTRODUCTION

Recently, the highlight of research in control for the automotive sector has been on autonomous vehicles, and the first results of much public and industrial funding have just started to appear on the road [1]. However, it will take years for these autonomous vehicles to supersede the existing personal transportation paradigm. In the meanwhile, we can expect road traffic to be composed of a mix of (i) vehicles fully controlled by a computer, (ii) human-driven vehicles capable of communication with other vehicles and infrastructure, and of autonomous actions under specific conditions, and (iii) vehicles without any autonomous capability. In the sequel, we will call (i) and (ii) *automated* vehicles, and call (iii) *legacy* vehicles.

In this paper, we design a control algorithm that modifies the inputs of automated vehicles only when necessary to avoid any collision at road intersections, while being able to cope with unpredictable behavior of legacy vehicles and the presence of measurement errors and unmodeled dynamics. If there is no predicted collision, the control algorithm leaves full control to the drivers. We call this algorithm a *least restrictive supervisor* [2–10]. In

[3,4], we presented methods to separately handle errors in measurement and dynamical model, and the presence of legacy vehicles, respectively. By merging the two methods, this paper presents a provably safe supervisor design that is robust to the simultaneous presence of measurement errors, unmodeled dynamics, and legacy vehicles. Such uncertainty sources were not considered in the other previous works [2,5–10].

The supervisor determines the time of overriding by checking at every time step the existence of an input signal that prevents future collisions. This is different from a controller in the autonomous vehicle paradigm, which coordinates vehicles by taking full control of them at all times. In such a paradigm, the collision avoidance problem for multiple vehicles is usually formulated as an optimization problem [11,12], where a performance metric can be fuel consumption [13,14], total travel time [15], or smooth speed change [16].

Because the algorithm that implements the supervisor has combinatorial complexity with the number of vehicles, we also present a novel strategy to design an approximate supervisor that can handle a larger number of vehicles in polynomially bounded time. Although the approximate

Manuscript received October 23, 2018; revised February 12, 2019 and June 3, 2019; accepted July 2, 2019. Recommended by Associate Editor Vu Nguyen under the direction of Editor Won-jong Kim. This work was done while Heejin Ahn was a Ph.D. candidate at the Massachusetts Institute of Technology.

Heejin Ahn is with Mitsubishi Electric Research Laboratories, 8th floor, 201 Broadway, Cambridge, MA 02139, USA (e-mail: heejin.ahn@alum.mit.edu). Andrea Rizzi is with Tri-Institutional Program in Computational Biology and Medicine, Weill Cornell Medical College, 1300 York Avenue, New York, NY 10065, USA (e-mail: ar963@cornell.edu). Alessandro Colombo is with Department of Electronics, Information, and Bioengineering, Politecnico di Milano, via Ponzio 34/5, 20133, Milano, Italy (e-mail: alessandro.colombo@polimi.it).

* Corresponding author.

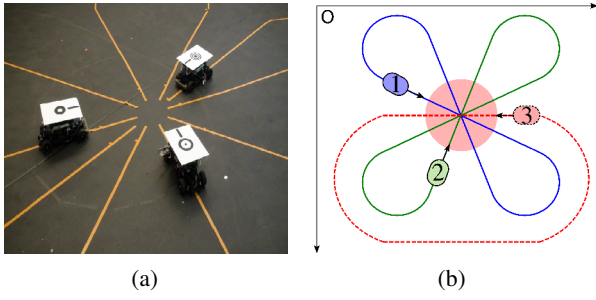


Fig. 1. Experimental setup. The supervisor modifies the inputs of cars 1 and 2 only when necessary to avoid collisions at the intersection (shaded circle) in the presence of a legacy vehicle (car 3).

supervisor is not least restrictive, we can quantify the approximation error bound. The approximate supervisor in this paper is less restrictive than the previously proposed approximate supervisors [3, 4]. That is, when the approximate supervisor in this paper overrides drivers, the previously proposed ones also override drivers. The converse is not always true.

We perform experimental validation of the least restrictive supervisor using three small-scale vehicles shown in Fig. 1, two of which are automated cars and the other is a legacy car. This experimental result was previously presented in [17] where the supervisor was designed only for three cars and thus the scalability or approximation of the supervisor was not discussed. This paper also presents a novel design of an approximate supervisor and validates it through simulation results. The experimental and simulation results demonstrate the practical feasibility of our method; they show that the supervisors prevent collisions even in the presence of uncertainty and that the approximate supervisor can be implemented in real-time even when a large number of vehicles are involved.

The rest of this paper is structured as follows: In Sections 2 and 3, we describe a test scenario and vehicle dynamics. We design a least restrictive supervisor and an approximate supervisor in Section 4, and present the simulation results in Section 5 and the experimental results in Section 6.

2. PROBLEM SETUP

We consider a set of roads leading to an intersection, populated by two types of vehicles: automated vehicles, which can be driven by a computer or by a human driver but have the capability of overriding their driver's input if necessary; and legacy vehicles, which are entirely driven by humans. For simplicity, we refer to the entity driving automated cars, be it a computer or a human, simply as the *driver*.

Let us consider $\bar{n}$ legacy and n automated vehicles ap-

proaching the intersection as in Fig. 2. We assume that the vehicles' paths through the intersection are known; this assumption hinges on the possibility to use road regulation that allows only one specific turn on each road, communication with vehicles' navigators, or inference algorithms to detect the vehicle's future path sufficiently early. Also, we assume that there is one vehicle per road. Thus, we design our algorithm to avoid side collisions, and can easily extend our algorithm to include rear-end collision avoidance by incorporating the method presented in [5]. The intersection is represented as an open interval (α_i, β_i) along the path of each vehicle i . Given the positions x_i and x_j of two vehicles, we say that they collide if

$$\exists t : x_i(t) \in (\alpha_i, \beta_i) \text{ and } x_j(t) \in (\alpha_j, \beta_j).$$

Note that the size of (α_i, β_i) is determined by the size of vehicles. Here, the intersection is modeled as a single conflict area and can be relaxed with some newly developed results, by separating multiple nearby conflict points [6, 7, 10], and decoupling supervisors on a network of intersections [9].

Our objective is to design a central coordinator, which can communicate with automated vehicles, verify safety of their input, and force them to apply a different input from that requested by their drivers if needed to avoid a collision. The coordinator can measure the positions and velocities of legacy vehicles (e.g. through radar, lidar, and cameras at the intersection), to enforce safety of the automated vehicles also against possible behavior of the legacy vehicles.

Our strategy is to develop a supervisor as in the framework of [18, 19]. The supervisor is a map

$$(\mathbf{x}, \mathbf{u}_d) \mapsto \mathbf{u}_a,$$

from a state $\mathbf{x}$ and a vector $\mathbf{u}_d$ of 'desired' control inputs that the drivers of automated vehicles request at a given time, to a vector $\mathbf{u}_a$ of inputs that the supervisor lets these

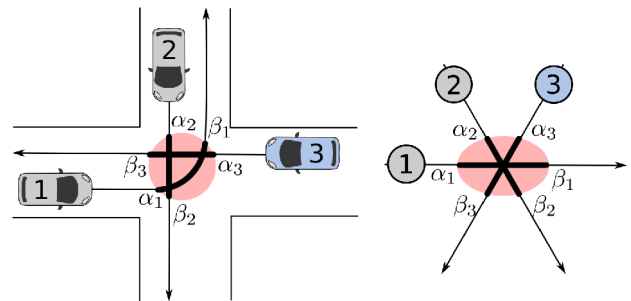


Fig. 2. Road intersection (left) and its simplified model (right). An intersection is modeled as an area containing the points at which the paths of vehicles intersect (the shaded area). Its location is denoted by (α_i, β_i) along the path of vehicle i .

vehicles apply. The objective of the supervisor is to let $\mathbf{u}_d = \mathbf{u}_a$, that is, to accept the input requested by the drivers unless it determines that this will cause an unavoidable future collision. A supervisor that can solve this problem exactly is called *least restrictive* [5].

3. MODEL AND ASSUMPTIONS

We design the supervisor to control longitudinal dynamics. We model the longitudinal dynamics of the vehicles as a second-order affine system with disturbances:

$$\begin{aligned} \dot{x}_i &= v_i + d_{x,i}, \\ \dot{v}_i &= \begin{cases} \max(0, a_i v_i + b_i + \phi_i u_i + d_{v,i}) & \text{if } (v_i = v_{i,\min}), \\ \min(0, a_i v_i + b_i + \phi_i u_i + d_{v,i}) & \text{if } (v_i = v_{i,\max}), \\ a_i v_i + b_i + \phi_i u_i + d_{v,i} & \text{otherwise.} \end{cases} \end{aligned} \quad (1)$$

The variables x_i and v_i are the position and velocity of vehicle i along its path, with velocity v_i bounded in the interval $[v_{i,\min}, v_{i,\max}] \subset [0, +\infty)$; $u_i \in [u_{i,\min}, u_{i,\max}] \subset \mathbb{R}$ is the control input; parameters a_i and b_i are a mass-normalized damping coefficient and friction; $d_{x,i}$ and $d_{v,i}$ are bounded disturbances, capturing the effect of imperfect path following, of the coupling of lateral and longitudinal dynamics [20], of the road slope, and of the engine's nonlinear behavior; ϕ_i is a gain dependent on the battery charge. In the following, we denote by $\mathbf{x}_i := (x_i, v_i)$ the full state of vehicle i and by $\mathbf{d}_i := (d_{x,i}, d_{v,i})$ its disturbance, bounded between $\mathbf{d}_{i,\min}$ and $\mathbf{d}_{i,\max}$. In this notation, the above equation is abbreviated to

$$\dot{\mathbf{x}}_i = f_i(\mathbf{x}_i, u_i, \mathbf{d}_i). \quad (2)$$

Legacy vehicles have the same dynamics, but neither the input u_i nor disturbance $\mathbf{d}_i$ is controllable.

We use the symbols $\mathbf{x} \in \mathbb{R}^{2n+2\bar{n}}$ and $\mathbf{d} \in \mathbb{R}^{2n+2\bar{n}}$ to denote the vectors of the aggregated states and disturbances of all vehicles, and the symbol $\mathbf{u} \in \mathbb{R}^n$ for the vector of control inputs of all the automated vehicles (we do not need to represent the aggregate inputs of legacy vehicles, though we will sometimes refer to the input u_i of a legacy car, even if it is an unmeasurable quantity). We call $\mathcal{U}$ the space of control input signals $\mathbf{u}$. We write $\mathbf{x}_i(t, u_i, \mathbf{d}_i, \mathbf{x}_{0,i})$ for the state reached at time t with inputs u_i and $\mathbf{d}_i$ assuming the initial condition $\mathbf{x}_{0,i}$. If $\mathbf{x}_{0,i} = \mathbf{x}_i(0)$, we omit the initial condition and write $\mathbf{x}_i(t, u_i, \mathbf{d}_i)$. With similar notation, $x_i(t, u_i, \mathbf{d}_i)$ is the position component of this state.

4. DESIGN OF SUPERVISORS

In this section, we describe the algorithmic structure of the least restrictive supervisor. To address its computational complexity, we also construct an approximate version of the supervisor which has polynomial-time complexity and guarantees a quantified approximation bound.

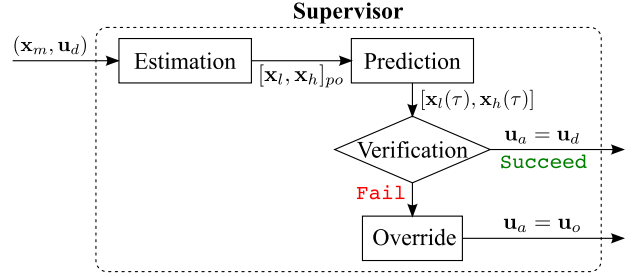


Fig. 3. Structure of the supervisor. The verification algorithm decides whether the supervisor should override the automated vehicles with an override input $\mathbf{u}_o$ to prevent collisions.

4.1. Least restrictive supervisor

The supervisor consists of an estimation algorithm, a state prediction algorithm, a verification algorithm, and an override input generation algorithm. These run in loop in short time steps of length τ (0.1 s in our experiment), as depicted in Fig. 3, to quickly adapt to the changing control inputs generated by drivers of automated vehicles, and to the unpredictable behavior of legacy vehicles.

We assume that the algorithms are synchronized among vehicles, and that communication takes a negligible time (asynchrony and communication delays can be handled with little overhead [21]). In the following, we detail the structure of each of the four steps of the supervisor loop: estimation, prediction, verification, and override input generation.

4.1.1 Estimation

At each time step, the state $\mathbf{x}_i$ of each automated vehicle is measured with bounded uncertainty δ_i , and communicated to the supervisor, together with the input $u_{d,i}$ issued by the driver. The driver can only change the input $u_{d,i}$ at the beginning of a supervisor time step; this is not an issue in practice, as long as the supervisor time step τ is sufficiently short. For the simulations and experiments in Sections V and VI, we use the supervisor time step of 0.1 s. The positions and velocities of legacy vehicles are likewise measured (e.g., via radar, lidar, and cameras at a roadside unit, which can communicate with the supervisor). This information is aggregated to form a vector $\mathbf{x}_m$ of the measured states of all vehicles, and a vector $\mathbf{u}_d$ of the inputs desired by drivers of all automated vehicles.

We call a *state estimate* the set of all states compatible with the system's measurements history at a given time. Consider the preorder $\mathbf{x} \leq \mathbf{y}$, meaning that each element of $\mathbf{x}$ is smaller than or equal to the corresponding element of $\mathbf{y}$. The dynamics of (2) are monotone in the control and disturbance inputs with respect to the above preorder [22]. This suggests to consider box state estimates, defined as

the set

$$\{\mathbf{x} : \mathbf{x}_l \leq \mathbf{x} \leq \mathbf{x}_h\},$$

and identified by their lower and upper bound, $[\mathbf{x}_l, \mathbf{x}_h]$. Given a prior state estimate $[\mathbf{x}_l, \mathbf{x}_h]_{pr}$, at each step the supervisor computes the posterior state estimate

$$[\mathbf{x}_l, \mathbf{x}_h]_{po} := [\mathbf{x}_l, \mathbf{x}_h]_{pr} \cap [\mathbf{x}_m + \delta_{\min}, \mathbf{x}_m + \delta_{\max}], \quad (3)$$

where $\delta_{\min}$ and $\delta_{\max}$ are the vectors of measurement uncertainty bounds. Because $[\mathbf{x}_l, \mathbf{x}_h]_{pr}$ and $[\mathbf{x}_m + \delta_{\min}, \mathbf{x}_m + \delta_{\max}]$ contain the actual state, the posterior state estimate (3) is nonempty.

4.1.2 Prediction

Set the current time $t = 0$, and consider a prediction interval of length τ equal to a supervisor time step. Given the current state estimate $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$, and a vector $\mathbf{u}_d$ of the control inputs issued by the driver of each automated vehicle at time 0, this algorithm predicts a box of possible future states $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]$. By exploiting monotonicity of the dynamics (2) with respect to input and disturbances, the algorithm propagates the lower and upper bound of the state estimate forward in time. For automated vehicle i , the control input $u_{d,i}$ in the interval $[0, \tau]$ is known, so the only uncertainty is due to the unknown disturbance input $\mathbf{d}_i$. The future state estimate is given by

$$\begin{aligned} & [\mathbf{x}_{l,i}(\tau), \mathbf{x}_{h,i}(\tau)] \\ & := [\mathbf{x}_{l,i}(\tau, u_{d,i}, \mathbf{d}_{i,\min}), \mathbf{x}_{h,i}(\tau, u_{d,i}, \mathbf{d}_{i,\max})]_{po}. \end{aligned}$$

For legacy vehicle i , due to the lack of information about the control inputs, uncertainty is both on control and disturbance inputs. Therefore, the future state estimate is given by

$$\begin{aligned} & [\mathbf{x}_{l,i}(\tau), \mathbf{x}_{h,i}(\tau)] \\ & := [\mathbf{x}_{l,i}(\tau, u_{i,\min}, \mathbf{d}_{i,\min}), \mathbf{x}_{h,i}(\tau, u_{i,\max}, \mathbf{d}_{i,\max})]_{po}. \end{aligned}$$

The future state estimate is used as the prior state estimate at the next time step.

4.1.3 Verification

Let us at first ignore the legacy vehicles, and consider the state prediction $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]$ returned by the prediction algorithm. The verification algorithm must check if the state prediction is compatible with a safe evolution of the system, that is, it must check the truth of the following proposition

$$\begin{aligned} & \exists \mathbf{u} \in \mathcal{U} : \\ & \forall i, j, \\ & \forall \mathbf{x}_{0,i} \in [\mathbf{x}_{l,i}(\tau), \mathbf{x}_{h,i}(\tau)], \forall \mathbf{x}_{0,j} \in [\mathbf{x}_{l,j}(\tau), \mathbf{x}_{h,j}(\tau)], \\ & \forall \mathbf{d}_i \in [\mathbf{d}_{i,\min}, \mathbf{d}_{i,\max}] \text{ and } \mathbf{d}_j \in [\mathbf{d}_{j,\min}, \mathbf{d}_{j,\max}], \end{aligned}$$

$$\begin{aligned} & \{t : x_i(t, u_i, \mathbf{d}_i, \mathbf{x}_{0,i}) \in (\alpha_i, \beta_i)\} \\ & \cap \{t : x_j(t, u_j, \mathbf{d}_j, \mathbf{x}_{0,j}) \in (\alpha_j, \beta_j)\} = \emptyset. \end{aligned} \quad (4)$$

If this is the case, the automated vehicles should be allowed to use the desired control input. Otherwise, the desired input should be overridden.

To verify the above proposition, the verification algorithm tests different vehicle orders, and solves for each order a sequence of optimal control problems. Let an *enumeration* of the vehicles be a map that attaches a unique number between 1 and n to each of n vehicles $j_1, \dots, j_n$:

$$(j_1, \dots, j_n) \rightarrow (1, \dots, n).$$

Given an enumeration of the vehicles, each vehicle has a unique number 1, 2, and so on.

Define $t_h(\alpha_i, u_i)$ as the time when $x_{h,i}$ reaches α_i using control input u_i and disturbance input $\mathbf{d}_{i,\max}$ with initial condition $\mathbf{x}_{h,i}(\tau)$. Similarly, define $t_l(\beta_i, u_i)$ as the time when $x_{l,i}$ reaches β_i using control input u_i and disturbance input $\mathbf{d}_{i,\min}$ with initial condition $\mathbf{x}_{l,i}(\tau)$. Define the optimal control problem

$$P_1 := \begin{cases} \min_{u_1} t_l(\beta_1, u_1) \\ \dot{\mathbf{x}}_{h,1} = f_1(\mathbf{x}_{h,1}, u_1, \mathbf{d}_{1,\max}) \\ u_1 \in [u_{1,\min}, u_{1,\max}], \end{cases} \quad (5)$$

for vehicle number 1, and the problems

$$P_i := \begin{cases} \min_{u_i} t_l(\beta_i, u_i) \\ \dot{\mathbf{x}}_{h,i} = f_i(\mathbf{x}_{h,i}, u_i, \mathbf{d}_{i,\max}) \\ \dot{\mathbf{x}}_{l,i-1} = f_{i-1}(\mathbf{x}_{l,i-1}, u_{i-1}, \mathbf{d}_{i-1,\min}) \\ u_i \in [u_{i,\min}, u_{i,\max}] \\ t_h(\alpha_i, u_i) \geq t_l(\beta_{i-1}, u_{i-1}), \end{cases} \quad (6)$$

for vehicles $2, \dots, n$. Verification succeeds if all of $P_1, \dots, P_n$, solved sequentially, have an optimal solution.

Legacy vehicles introduce another set of constraints: for each automated vehicle i and legacy vehicle k ,

$$\begin{aligned} & \forall \mathbf{x}_{0,k} \in [\mathbf{x}_{l,k}(\tau), \mathbf{x}_{h,k}(\tau)], \\ & \forall u_k \in [u_{k,\min}, u_{k,\max}] \text{ and } \mathbf{d}_k \in [\mathbf{d}_{k,\min}, \mathbf{d}_{k,\max}], \\ & \{t : x_i(t, u_i, \mathbf{d}_i, \mathbf{x}_{0,i}) \in (\alpha_i, \beta_i)\} \\ & \cap \{t : x_k(t, u_k, \mathbf{d}_k, \mathbf{x}_{0,k}) \in (\alpha_k, \beta_k)\} = \emptyset. \end{aligned} \quad (7)$$

Thus, in the presence of legacy vehicles, the above constraints must be added to (4). To account for this additional set of constraints on $P_1, \dots, P_n$, we compute for each legacy vehicle k the interval $[t_h(\alpha_k, u_{k,\max}), t_l(\beta_k, u_{k,\min})]$, and add to each of $P_1, \dots, P_n$ the constraint

$$\begin{aligned} & [t_h(\alpha_i, u_i), t_l(\beta_i, u_i)] \\ & \cap [t_h(\alpha_k, u_{k,\max}), t_l(\beta_k, u_{k,\min})] = \emptyset, \forall i. \end{aligned} \quad (8)$$

This idea was first proposed in [4] (for a scenario without disturbances and measurement uncertainty), and ensures

that the input vector chosen for the automated vehicles will not bring any of these in the intersection at a time when a legacy vehicle might be occupying it.

We have the following equivalence result:

Theorem 1: There exists an enumeration $1, \dots, n$ of vehicles such that $P_1, \dots, P_n$ admits a solution if and only if there exists an input $\mathbf{u}$ satisfying (4) and (7).

Proof: It is a consequence of Theorem 1 in [3] (in the absence of legacy vehicles) and in [4] (in the absence of disturbances). $\square$

We say that an enumeration is *admissible* if all of $P_1, \dots, P_n$ solved sequentially according to the enumeration, have an optimal solution.

4.1.4 Override input generation

When the verification algorithm predicts that some states in the future state estimate, a set of states possible to reach at time τ with the desired input $\mathbf{u}_d$, would inevitably lead to a collision, an override input must be computed and applied at time 0. In practice, the override input at time 0 can be obtained as the optimal solution of $P_1, \dots, P_n$ for an admissible enumeration, using $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ as initial conditions in $P_1, \dots, P_n$. If there are more than one optimal solutions, any one solution can be chosen as the override input. When the existence of such an override input is guaranteed, the supervisor is said to be *nonblocking*.

Algorithm 1 implements the supervisor. If there exists an admissible enumeration to $P_1, \dots, P_n$ using $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]$ as initial conditions (line 5), then the supervisor allows the desired input $\mathbf{u}_d$ (line 6). Otherwise, the supervisor overrides the automated vehicles with an override input $\mathbf{u}_o$ (line 10). Finding an admissible enumeration in line 4 may require exploring all possible enumerations. Thus, this algorithm has combinatorial complexity with the number of automated vehicles. To address this issue, we present an approximate supervisor in the next section.

Algorithm 1: Least restrictive supervisor.

- 1: **Estimation:** Estimate $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$
 - 2: **Prediction:** Given $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ and $\mathbf{u}_d$, predict $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]$.
 - 3: **Verification:**
 - 4: Find an enumeration that is admissible
 - 5: **if** there exists such an enumeration **then**
 - 6: **return** $\mathbf{u}_a = \mathbf{u}_d$
 - 7: **else**
 - 8: **Override input generation:**
 - 9: Solve $P_1, \dots, P_n$ with an admissible enumeration and initial condition $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ to generate an override input $\mathbf{u}_o$
 - 10: **return** $\mathbf{u}_a = \mathbf{u}_o$
-

Theorem 2: The supervisor in Algorithm 1 guarantees safety and is least restrictive. Also, it is nonblocking.

Proof: It is a consequence of Theorem 3 in [3] and Theorem 2 in [4]. $\square$

4.2. Approximate supervisor

Even though the least restrictive supervisor is theoretically computable, exploring all possible enumerations for a large number of vehicles is typically too slow. Thus, we need a novel approach to find a good candidate enumeration quickly, which enables the construction of an approximate supervisor in this section.

Our approach can be constructed based on two observations:

- 1) Finding an enumeration for which $P_1, \dots, P_n$ has an optimal solution is equivalent to solving a scheduling problem, where a vector of optimal values $t_h(\alpha_i, u_i)$ for all automated vehicles is a schedule and the constraints are functions of the constraints of $P_1, \dots, P_n$ [3, 4].
- 2) The problem of scheduling jobs of the same length has an exact solution with polynomial complexity [23].

We propose a modification to the approaches of [3, 4] and use the problem of scheduling jobs of the same length, in which jobs have the same process time [23], to extract a candidate enumeration. The approximate supervisors presented in those works are highly restrictive, due to the way the verification and override input generation steps are designed. We provide here a sharp improvement to those results, which uses a separation between the computation of the enumeration and the generation of a feasible schedule and an override input.

We begin by fixing a suitably large constant θ (details on the size of θ are found in [3]), which is the process time of jobs, and defining $R_i := t_h(\alpha_i, u_{i,\max})/\theta$ and $D_i := t_h(\alpha_i, u_{i,\min})/\theta + 1$ for each automated vehicle i . We call $\mathbf{R}$ and $\mathbf{D}$ the vectors of R_i and D_i , and call σ a vector of indices in the increasing order of R_i (so that $R_{\sigma_1} \leq R_{\sigma_2} \leq \dots$). Furthermore, we define a set of intervals

$$\mathbf{F} := \{F_k = [t_h(\alpha_k, u_{k,\max})/\theta - 1, t_l(\beta_k, u_{k,\min})/\theta], \\ \forall \text{ legacy vehicle } k\}.$$

Then, we use Algorithm 2 to compute an enumeration. It is a modified version of the scheduling algorithm in [23]. The basic idea of the algorithm relies on the concept of a forbidden region, a set of time intervals in which jobs are forbidden to be scheduled. As part of forbidden region declaration, the algorithm determines the feasibility of a schedule of same-length jobs (line 8). Then, one with smallest D_i among the jobs is scheduled earlier than the others while avoiding the forbidden region (line 15). This

Algorithm 2: Modified version of the result of [23].

```

1: procedure COMPUTE ENUMERATION( $\mathbf{R}, \mathbf{D}, \mathbf{F}$ )
2:   Forbidden region declaration:
3:   for  $i = n$  to 1 do
4:     for  $j \in \{j : D_j \geq D_{\sigma_i}\}$  do
5:       if  $c_j$  undefined then  $c_j = D_j$  else  $c_j = c_j - 1$ 
6:       if  $c_j \in F_\gamma$  for some  $\gamma$  then  $c_j = \inf F_\gamma$ 
7:       if  $\sigma_i = 1$  or  $R_{\sigma_{i-1}} < R_{\sigma_i}$  then  $c = \min\{c_j \text{ defined}\}$ 
8:       if  $c < R_{\sigma_i}$  then a feasible schedule does not exist
        in the scheduling problem with same-length jobs
9:       if  $R_{\sigma_i} \leq c < R_{\sigma_i} + 1$  then  $\mathbf{F} = \mathbf{F} \cup (c - 1, R_{\sigma_i})$ 
10:  Enumeration computation:
11:   $s = 0, \mathcal{A} = \emptyset, \mathcal{B} = \{1, \dots, |\mathbf{R}|\}$ 
12:  while  $\mathcal{B} \neq \emptyset$  do
13:    if  $\mathcal{A} = \emptyset$  then  $s = \min_{i \in \mathcal{B}} R_i$ 
14:    if  $s \in F_\gamma$  for some  $\gamma$  then  $s = \sup F_\gamma$ 
15:     $\mathcal{A} = \{k \in \mathcal{B} : R_k \geq s\}$  and  $j = \arg \min_{i \in \mathcal{A}} D_i$ 
16:     $\mathbf{t}_j = s, s = s + 1$ , and  $\mathcal{B} = \mathcal{B} \setminus \{j\}$ 
17:   $\pi^* :=$  an enumeration in increasing order of  $\mathbf{t} =$ 
     $(\mathbf{t}_1, \dots, \mathbf{t}_n)$ 
18:  return  $\pi^*$ 

```

scheduling algorithm is proved to yield the exact solution to the scheduling problem with same-length jobs. Algorithm 2 is a straightforward variation of the scheduling algorithm proposed in [23], changed so as to return an enumeration π^* even if the scheduling problem does not admit a feasible schedule $\mathbf{t}$. The detailed explanation of the algorithm is referred to [23].

We then set up the supervisor as in Algorithm 3. The approximate supervisor in Algorithm 3 checks if the enumeration that Algorithm 2 returns is admissible (line 6), and records an admissible enumeration at each time step (line 7). Note that Algorithm 3 has a polynomial-time complexity.

Theorem 3: The supervisor in Algorithm 3 guarantees safety and is nonblocking.

Proof: (Sketch) The supervisor in Algorithm 3 leaves full control to the drivers when there exists an admissible enumeration, which implies by Theorem 1 that the drivers' inputs will not result in an unavoidable future collision. When the supervisor overrides the drivers, it applies an override input that guarantees safety in the future. We need to prove that such an override input $\mathbf{u}_o$ exists at every time step of the supervisor. This is the nonblocking property.

Let $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]^*$ denote the state prediction at the previous step. Suppose the enumeration π was obtained in line 6 at the previous step. The estimate $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ at the current step is $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]^*$ compatible with new measurement, that is, $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po} \subseteq [\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]^*$ by

Algorithm 3: Approximate supervisor.

```

1: Estimation: Estimate  $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ 
2: Prediction: Given  $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$  and  $\mathbf{u}_d$ , predict
    $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]$ .
3: Verification:
4: Compute  $\mathbf{R}, \mathbf{D}$ , and  $\mathbf{F}$ 
5:  $\pi^* = \text{COMPUTE ENUMERATION}(\mathbf{R}, \mathbf{D}, \mathbf{F})$ 
6: if enumeration  $\pi^*$  is admissible then
7:   Record enumeration  $\pi^*$  as  $\pi$  for use at next iteration
8:   return  $\mathbf{u}_a = \mathbf{u}_d$ 
9: else
10:  Override input generation:
11:  Solve  $P_1, \dots, P_n$  with enumeration  $\pi$  and initial condition
     $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$  to generate an override input  $\mathbf{u}_o$ 
12:  Keep  $\pi$  unchanged for next iteration
13:  return  $\mathbf{u}_a = \mathbf{u}_o$ 

```

(3). Note that the fact that $P_1, \dots, P_n$ was feasible for initial condition $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]^*$ implies that it is feasible for $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ with the same enumeration, because given an input the state evolution of $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ is contained in the state evolution of $[\mathbf{x}_l(\tau), \mathbf{x}_h(\tau)]^*$.

Next, suppose the enumeration π was unchanged in line 12 at the previous step. Let $\mathbf{u}_o^*$ and $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}^*$ denote the override input and the given estimate at the previous step. Then the initial condition at the current step satisfies $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po} \subseteq [\mathbf{x}_l(\tau, \mathbf{u}_o^*, \mathbf{d}_{\min}), \mathbf{x}_h(\tau, \mathbf{u}_o^*, \mathbf{d}_{\max})]_{po}^*$. With the same observation as before, $P_1, \dots, P_n$ for initial condition $[\mathbf{x}_l(0), \mathbf{x}_h(0)]_{po}$ is feasible. $\square$

The theorem above shows that an override input that prevents any future collision exists at each time step of the approximate supervisor, and the approximate supervisor guarantees safety.

Algorithm 3 is less restrictive than the previously proposed algorithms [3], [4]. Algorithm 3 overrides the drivers when an enumeration obtained from Algorithm 2 is not admissible to the optimal control problems $P_1, \dots, P_n$ (line 6). In contrast, the previously proposed algorithms override the drivers when a feasible schedule does not exist in the scheduling problem with same-length jobs (line 8 in Algorithm 2). Note that a feasible schedule of jobs that have the same process time θ is also a feasible schedule of jobs that have process times shorter than or equal to θ . Such a feasible schedule implies the existence of an admissible enumeration from the observations at the beginning of this section. Therefore, a feasible schedule of same-length jobs does not exist whenever there is no admissible enumeration to $P_1, \dots, P_n$. That is, the cases when Algorithm 3 returns an override input are strictly included in the cases when the algorithms in [3], [4] do so.

Although Algorithm 3 is less restrictive than the algorithm in [3], both algorithms have the same worst-case ap-

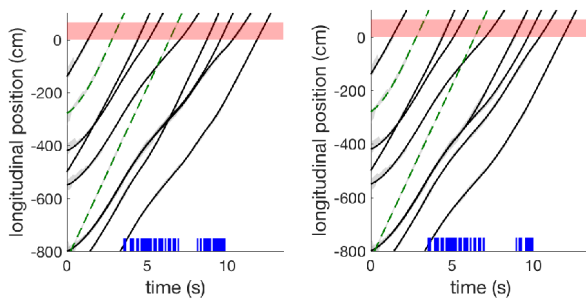
proximation error. The approximation error bound can be found in [3].

5. SIMULATIONS

In this section, we present the results of computer simulations of Algorithm 1 (least restrictive supervisor) and Algorithm 3 (approximate supervisor). The simulations were performed using MATLAB on a personal computer with an 3.10 GHz Intel Core i7-3770s processor with 8 GB RAM, with control gain $\phi_i = 1$ and desired input $u_{d,i} = 150$ for all i . The values of the other parameters are presented in the next section.

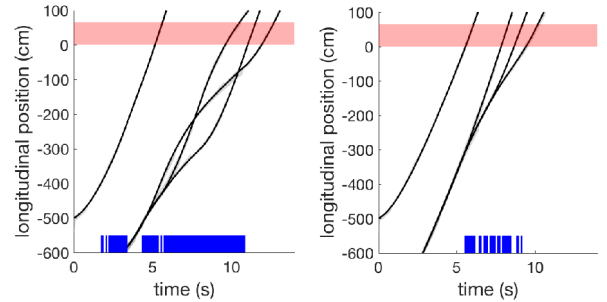
In Fig. 4, we show a run of Algorithm 1 and Algorithm 3 on the same initial conditions, desired input signals, and disturbance signals in a scenario with eight automated vehicles and two legacy vehicles. Note that the supervisor overrides the automated vehicles (the blue bands appear) to avoid a collision at the intersection at (0, 65). This confirms Theorems 2 and 3. The supervisor overrides the vehicles for as many time intervals of length τ as it is needed to avert collisions. Each iteration of Algorithm 1 was executed in 2.92 s, and that of Algorithm 3 was executed in less than 0.007 s. The approximate supervisor performs close to the least restrictive supervisor in simulations.

To compare Algorithm 3 with the algorithm in [3], we have tested a scenario with four automated vehicles and no legacy vehicle (which the algorithm in [3] could not handle), running the two algorithms with the same initial conditions, desired input signals, and disturbance signals. The



(a) Least restrictive supervisor. (b) Approximate supervisor.

Fig. 4. The solid black and dotted green lines represent the actual position of the automated and legacy vehicles, respectively, with gray surrounding representing the state estimate. The blue bands at the bottom appear when the supervisor overrides the automated vehicles to prevent a collision at the intersection (red band at (0, 65)). As indicated by the blue bands, the approximate supervisor takes control away from the drivers just as frequently as the least restrictive supervisor.



(a) Supervisor in [3]. (b) Supervisor in Algorithm 3.

Fig. 5. The blue bands at the bottom show that our supervisor in (b) starts an override later than the supervisor in (a). The same coloring is used as in Fig. 4.

blue bands at the bottom of Fig. 5 indicate times when the supervisor is overriding the drivers' desired input; the supervisor from [3], in the left panel, overrides much earlier than Algorithm 3, in the right panel. We also performed 100 tests with random initial conditions of four vehicles. The average ratio of the number of overrides to the number of steps until all vehicles exit the intersection (set to be constant at 200 steps) is 0.42 for the algorithm in [3] and 0.09 for Algorithm 3. This confirms that Algorithm 3 indeed restricts the drivers much less than the supervisor in [3].

6. EXPERIMENTAL RESULTS

We have implemented Algorithm 1 (least restrictive supervisor), which runs with a time step of $\tau = 0.1$ s, in an intersection testbed. As shown in Fig. 1(a), we use three small-scale custom built robots on Tamiya TT-01R car chassis. The motor and steering servo are controlled by a microcontroller (Acroname Moto 1.0), and the control inputs are commanded by an on-board computer (Mini-ITX running Linux Fedora Core). The positions and velocities of all cars are measured by an overhead camera system (six 640×480 BW Firefly MV cameras) and a quadrature encoder mounted on the rear wheel axle of each car. The centralized computer collects the position and velocity data of all the cars, and the desired input of automated cars (cars 1 and 2), via 802.11b wireless communication. The supervisor algorithm, which is implemented on the on-board computer of each automated car, receives this collection of data and computes inputs to the automated cars (i.e., $\mathbf{u}_a$). Because the supervisor algorithm returns the same $\mathbf{u}_a$ for the same data, this implementation is equivalent to the supervisor running on the centralized computer and communicating its result to the automated cars. Car 3 is legacy, and is remotely controlled by a human operator. Each car follows one of the three paths in Fig. 1(b), which intersect at the center. For the details of

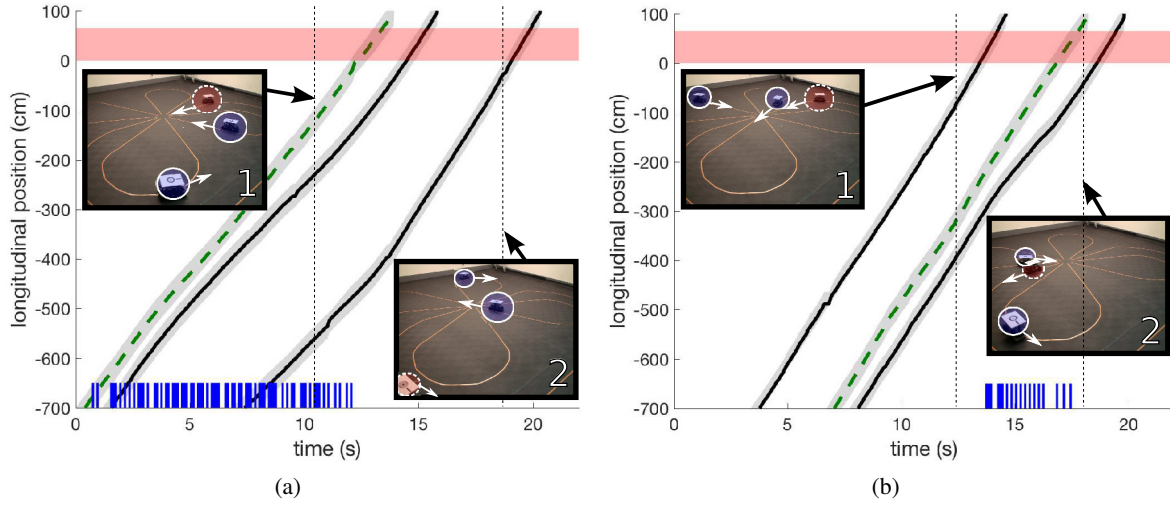


Fig. 6. Experimental results of the supervisor. The solid black and dotted green lines represent the position measurement of the automated and legacy cars, respectively. The same coloring is used as in Fig. 4.

the implementation, readers are referred to [17, 24].

We have investigated the sources of disturbances $\mathbf{d}_i = (d_{x,i}, d_{v,i})$ for each car i and designed a compensating input to reduce their effects. For the disturbance input on the position dynamics, $d_{x,i} = d_{proj,i}$, where $d_{proj,i}$ is the effect of the car's wiggling motion about its nominal path on the longitudinal velocity. For the disturbance input on the speed dynamics,

$$d_{v,i} := d_{power,i} + d_{slope,i} + d_{steer,i},$$

where $d_{power,i}$, $d_{slope,i}$, and $d_{steer,i}$ are the effects on the longitudinal acceleration of the engine's nonlinear behavior, of the road slope, and of the lateral-longitudinal dynamics coupling, respectively. We model $d_{power,i} = g_i e^{-t/h_i} u_i$ where g_i, h_i are a gain and a time constant, and u_i is the motor input. The disturbances $d_{slope,i}$ and $d_{steer,i}$ vary on the position x_i and are estimated off-line. The effects of $d_{steer,i}$, $d_{slope,i}$, and $d_{power,i}$ are reduced through the introduction of a compensating input $c_i(t, x_i)$

$$c_i(t, x_i) = -\frac{g_i e^{-t/h_i} + d_{slope,i}(x_i) + d_{steer,i}(x_i)}{g_i e^{-t/h_i} + \phi_i}.$$

By applying an input $u_i = u'_i + c_i(t, x_i)$, the car dynamics become

$$\dot{v}_i = a_i v_i + b_i + \phi_i u'_i + d'_{v,i},$$

where $d'_{v,i} = g_i e^{-t/h_i} (u'_i - 1)$ is the effective disturbance after the compensation. The gain ϕ_i in our experiments depends on the battery charge and is estimated online using adaptive control.

Through a set of experiments on each car, we have identified the values for the parameters, which are given as follows: $a = (-0.53, -0.30, -0.43) \text{ s}^{-1}$, and $b = (-84.68, -66.43,$

$-49.64) \text{ cm/s}^2$. The bounds of the speed are $v_{\min} = (25, 25, 40) \text{ cm/s}$ and $v_{\max} = (200, 200, 130) \text{ cm/s}$, and those of the motor input u'_i (PWM) are $u_{\min} = (105, 105, 130)$ and $u_{\max} = (170, 165, 150)$. The bounds for disturbance inputs and measurement errors are chosen from their distributions obtained from experiments. The bounds for d_x are $(-5, -3, -3) \text{ cm/s}$ and $(3, 4, 3) \text{ cm/s}$ and for d'_v , $(-4, -2, -3) \text{ cm/s}^2$ and $(2, 3, 2.5) \text{ cm/s}^2$. The measurement error bounds are $\delta_{i,\min} = (-25 \text{ cm}, -25 \text{ cm/s})$ and $\delta_{i,\max} = (25 \text{ cm}, 16 \text{ cm/s})$ for all i . For computer simulations described in the previous section, the parameter values of car 1 and car 3 are used for automated and legacy vehicles, respectively.

Fig. 6 depicts two experimental results near the intersection located at $(0, 65) \text{ cm}$ for all cars. In the first scenario (a), panel 1 shows that the legacy car (dotted red circle) approaches the intersection earlier than the automated cars (solid blue circles). Thus, the supervisor forces the automated cars to decelerate to avoid a conflict. Panel 2 shows that the conflict is resolved, and one automated car is crossing the intersection alone without overrides. In the second scenario (b) as shown in panel 1, all three cars are close to the intersection, and the supervisor forces one automated car to accelerate and the other to decelerate to prevent a collision with the legacy car. Note that the upper state estimate of the last car's position enters the intersection right after the lower state estimate of the legacy car's position has exited, indicating that the override was necessary to avoid the collision. In both scenarios, intersection collisions are averted.

Fig. 7 depicts 509 trajectories (semitransparent black lines) near the set in which the cars collide (red solid). From the 2D projections in (b), we can confirm that none of the trajectories enters the red solid. We can see that the supervisor overrides cars 1 and 2 when the trajectories

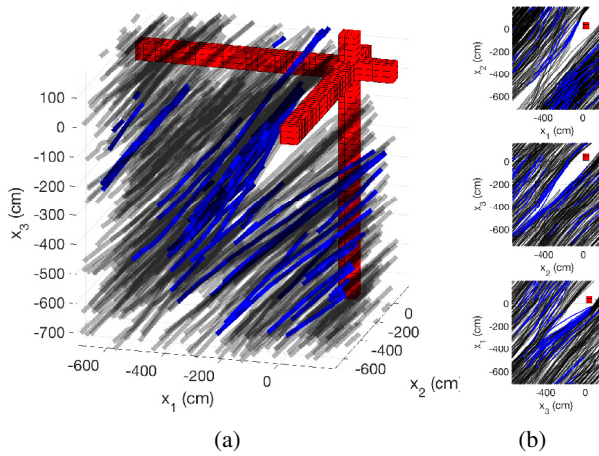


Fig. 7. Position trajectories of cars 1-3. The red region represents a set of positions where cars collide. The trajectories (semitransparent black lines) are in blue when the supervisor intervenes. The 2D projections confirm that no trajectory enters the red region.

would enter the red solid if the trajectories were straight lines. We observed the failure of 15 out of 509 experiments due to disturbances and measurement errors outside the model bounds. In the 15 experiments, the supervisor terminates before the intersection because of incompatibility of the state estimate and measurement. The choice of small bounds was necessary in the confined laboratory because otherwise a feasible initial condition may not always exist.

7. CONCLUSIONS

We presented the design of a supervisor that takes control of automated vehicles only when necessary to prevent a collision, in the simultaneous presence of legacy vehicles, measurement errors, and unmodeled dynamics. To do this, we formulated a sequence of optimal control problems that checks whether there exists an unavoidable future collision for some uncertainty and yields an override input of the supervisor. We also presented an approximate supervisor, by devising a heuristic to the optimal control problems, that can handle a greater number of vehicles in polynomially bounded time in a quantified approximation bound. We performed computer simulations and experimental testings to validate that the supervisors override vehicles to prevent them from colliding for any uncertainty. Also, the simulations showed that the approximate supervisor takes only 0.007 s to handle eight automated and two legacy vehicles whereas the least restrictive supervisor takes 2.92 s. Future work includes the incorporation of inference algorithms that enable less conservative prediction of legacy vehicles' future behavior and vehicle paths through intersections.

REFERENCES

- [1] "Waymo racks up 10 million test miles ahead of launching its robotaxi business," <https://www.forbes.com/sites/alanohnsman/2018/10/10/waymo-racks-up-10-million-test-miles-ahead-of-launching-its-robotaxi-business/>, 2018.
- [2] A. Colombo and D. Del Vecchio, "Efficient algorithms for collision avoidance at intersections," *Proc. of the Int. Conf. Hybrid Systems: Computation and Control*, pp. 145-154, 2012.
- [3] L. Bruni, A. Colombo, and D. Del Vecchio, "Robust multi-agent collision avoidance through scheduling," *Proc. of the IEEE Conf. Decision and Control*, pp. 3944-3950, 2013.
- [4] H. Ahn, A. Colombo, and D. Del Vecchio, "Supervisory control for intersection collision avoidance in the presence of uncontrolled vehicles," *Proc. of the American Control Conference*, p. 867-873, 2014.
- [5] A. Colombo and D. Del Vecchio, "Least restrictive supervisors for intersection collision avoidance: A scheduling approach," *IEEE Trans. on Automatic Control*, vol. 60, no. 6, pp. 1515-1527, June 2015.
- [6] H. Ahn, and D. Del Vecchio, "Semi-autonomous intersection collision avoidance through job-shop scheduling," *Proc. of the Int. Conf. Hybrid Systems: Computation and control*, pp. 185-194, 2016.
- [7] F. Alth  , X. Qian, and A. de La Fortelle, "Least restrictive and minimally deviating supervisor for safe semi-autonomous driving at an intersection: An MIQP approach," *Proc. of the Int. Conf. Intelligent Transportation Systems*, pp. 2520-2526, 2016.
- [8] F. Alth  , X. Qian, and A. de La Fortelle, "An algorithm for supervised driving of cooperative semi-autonomous vehicles," *IEEE Trans. on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3527-3539, September 2017.
- [9] A. Colombo, G. R. De Campos, and F. Della Rossa, "Control of a city road network: Distributed exact verification of traffic safety," *IEEE Trans. on Automatic Control*, vol. 62, no. 10, pp. 4933-4948, October 2017.
- [10] H. Ahn and D. Del Vecchio, "Safety verification and control for collision avoidance at road intersections," *IEEE Trans. on Automatic Control*, vol. 63, no. 3, pp. 630-642, March 2018.
- [11] L. Chen and C. Englund, "Cooperative intersection management: a survey," *IEEE Trans. on Intelligent Transportation Systems*, vol. 17, no. 2, pp. 570-586, September 2015.
- [12] N. Murgovski, G. R. de Campos, and J. S  berg, "Convex modeling of conflict resolution at traffic intersections," *Proc. of the IEEE Conf. Decision and Control*, pp. 4708-4713, 2015.
- [13] A. Richards, T. Schouwenaars, J. P. How, and E. Feron, "Spacecraft trajectory planning with avoidance constraints using mixed-integer linear programming," *J. Guidance, Control, and Dynamics*, vol. 25, no. 4, pp. 755-764, July 2002.

- [14] J. Omer, "A space-discretized mixed-integer linear model for air-conflict resolution with speed and heading maneuvers," *Computers & Operations Research*, vol. 58, pp. 75-86, June 2015.
- [15] L. Pallottino, E. Feron, and A. Bicchi, "Conflict resolution problems for air traffic management systems solved with mixed integer programming," *IEEE Trans. on Intelligent Transportation Systems*, vol. 3, no. 1, pp. 3-11, August 2002.
- [16] M. Kamal, J. Imura, T. Hayakawa, A. Ohata, and K. Aihara, "A vehicle-intersection coordination scheme for smooth flows of traffic without using traffic lights," *IEEE Trans. on Intelligent Transportation Systems*, vol. 16, no. 3, pp. 1136-1147, June 2015.
- [17] H. Ahn, A. Rizzi, A. Colombo, and D. Del Vecchio, "Experimental testing of semi-autonomous multi-vehicle control for collision avoidance at intersections," *Proc. of the IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, pp. 4834-4839, 2015.
- [18] C. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, Springer Science & Business Media, 2009.
- [19] P. J. Ramadge and W. M. Wonham, "Supervisory control of a class of discrete event processes," *SIAM J. Control and Optimization*, vol. 25, no. 1, pp. 206-230, January 1987.
- [20] P. Falcone, M. Tufo, F. Borrelli, J. Asgari, and H. E. Tseng, "A linear time varying model predictive control approach to the integrated vehicle dynamics control problem in autonomous systems," *Proc. of the IEEE Conf. Decision and Control*, pp. 2980-2985, 2007.
- [21] D. Bresch-Pietri and D. Del Vecchio, "Estimation for decentralized safety control under communication delay and measurement uncertainty," *Automatica*, vol. 62, pp. 292-303, December 2015.
- [22] D. Angeli and E. Sontag, "Monotone control systems," *IEEE Trans. on Automatic Control*, vol. 48, no. 10, pp. 1684-1698, October 2003.
- [23] M. R. Garey, D. S. Johnson, B. B. Simons, and R. E. Tarjan, "Scheduling unit-time tasks with arbitrary release times and deadlines," *SIAM J. Computing*, vol. 10, no. 2, pp. 256-269, May 1980.
- [24] R. Verma, D. Del Vecchio, and H. K. Fathy, "Development of a scaled vehicle with longitudinal dynamics of an HMMWV for an ITS testbed," *IEEE/ASME Trans. on Mechatronics*, pp. 46-57, February 2008.



systems with application to transportation systems.

Heejin Ahn received her B.S. degree from Seoul National University in 2012, and her S.M. and Ph.D. degrees from the Massachusetts Institute of Technology (MIT), in 2014 and 2018, respectively. She is currently a visiting research scientist at Mitsubishi Electric Research Laboratories. Her research interests include the analysis and control of networked dynamical systems with application to transportation systems.



centered on improving molecular models for the prediction of small molecule-protein binding.

Andrea Rizzi obtained both his Laurea and Laurea Magistrale in Engineering of Computing Systems from Politecnico di Milano, in 2010 and 2014, respectively. He is currently a Ph.D. candidate in the Tri-Institutional Ph.D. Program in Computational Biology & Medicine in Dr. Chodera's lab at Memorial Sloan Kettering Cancer Center. His research is centered on improving molecular models for the prediction of small molecule-protein binding.



His research interests are in the analysis and control of discontinuous and hybrid systems with applications, among others, to transportation networks.

Alessandro Colombo received his Diplôme D'Ingénieur from ENSTA in Paris in 2005, and a Ph.D. from Politecnico di Milano in 2009. He was a Post-doctoral Associate at the Massachusetts Institute of Technology in 2010-2012, and is currently an associate professor in the Department of Electronics, Information and Bioengineering at Politecnico di Milano.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.